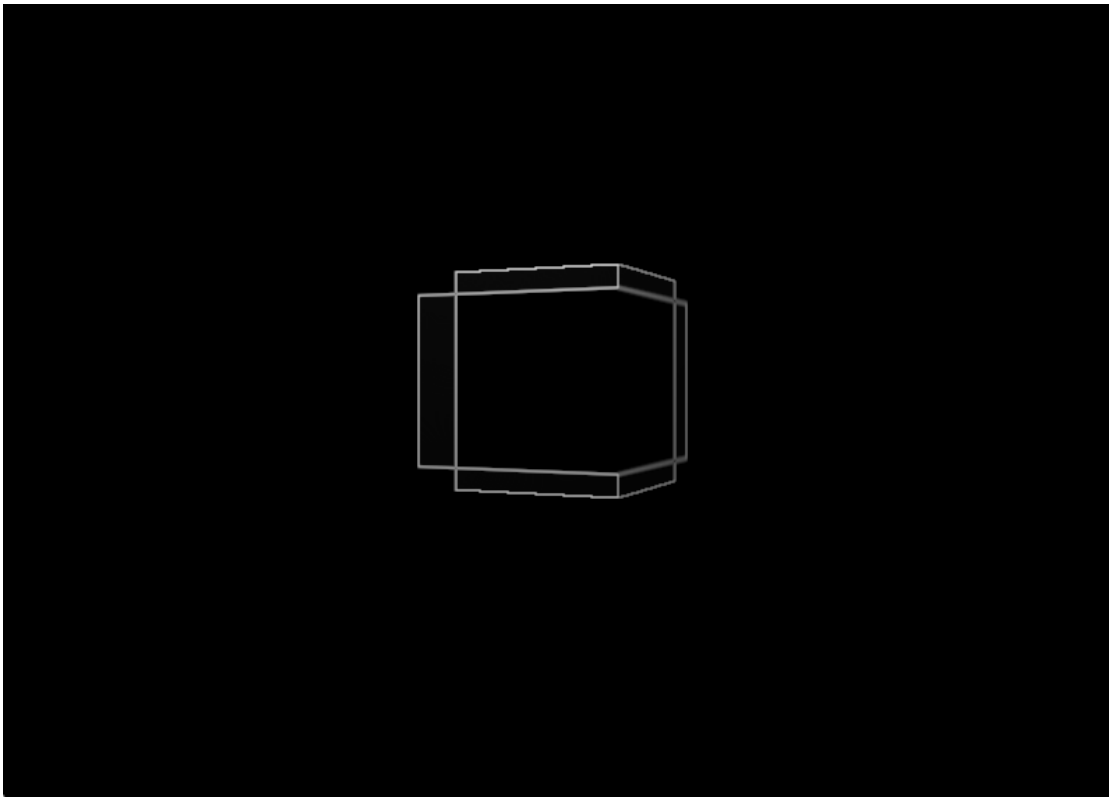
Edge Detection in Games: Comparing Sobel and Laplacian Shaders

By

Stephan Gibson



Submitted to Saxion University of Applied Science Enschede

In fulfilment for the course Advanced Rendering

April 2026

Abstract

In modern game development, visual effects play a huge role in shaping how players experience a game. From stylized outlines to realistic post-processing, edge detection is often used to enhance visuals, improve readability, or create unique artistic styles. With the power of GPUs, these effects are commonly implemented using shaders, allowing them to run in real time without heavily impacting performance. Two popular techniques used for edge detection in shaders are the Sobel operator and the Laplacian operator.

The Sobel shader works by detecting changes in brightness across horizontal and vertical directions, making it especially useful for outlining objects and enhancing edges in a controlled, visually clear way. On the other hand, the Laplacian shader focuses on detecting rapid changes in intensity from all directions at once. This can produce sharper and more detailed edges, which can be useful for certain visual styles, but it also tends to pick up more noise and can look harsher if not handled carefully.

In this paper, we compare Sobel and Laplacian shaders specifically in the context of game development. We look at how they perform in real-time rendering, how they affect visual style, and where each one works best. The goal is to better understand which technique is more suitable depending on the type of game and the visual effect you're trying to achieve.

Edge detection

Edge detection is a technique used in image processing to identify areas in an image where there is a sharp change in brightness or color. These changes usually represent the boundaries of objects, surface details, or important features within a scene (Gonzalez & Woods, 2018). In games, edge detection helps highlight the structure of objects, making scenes easier to read visually or enabling specific visual styles.

Instead of processing full textures or complex geometry, edge detection focuses only on where significant changes occur. This makes it useful for effects like outlining objects, enhancing contrast, or creating stylized visuals such as toon shading (Akenine-Möller et al., 2018). By detecting edges, developers can emphasize important elements in a scene without adding extra geometry or models.

Shaders in games

Shaders are small programs that run on the GPU and control how things are rendered on the screen. In games, they are used to calculate things like color, lighting, shadows, and special effects in real time (Akenine-Möller et al., 2018). Because they run on the GPU, shaders are highly efficient and can process many pixels at once.

For edge detection, a specific type called a fragment shader is typically used. This shader processes each pixel on the screen and can sample neighboring pixels to detect differences in color or brightness (Rost et al., 2014). By comparing these values, the shader determines whether a pixel is part of an edge and adjusts its color accordingly.

This approach allows edge detection effects, such as Sobel and Laplacian filters, to be applied in real time during gameplay. As a result, games can achieve advanced visual effects without significantly impacting performance (Akenine-Möller et al., 2018).

Sobel operator

The Sobel operator is an edge detection technique that works by measuring how quickly image intensity changes in different directions. It does this by applying two small filters (kernels) to an image: one detects changes horizontally, and the other detects changes vertically (Gonzalez & Woods, 2018). By combining these two results, the shader can estimate the strength of edges at each pixel.

In practice, the Sobel shader samples the surrounding pixels in a 3×3 grid and calculates gradients along the x and y axes. These gradients represent how much the brightness changes in each direction. The final edge strength is then computed by combining these values, usually resulting in smoother and more continuous edges (Sobel, 1970).

Because the Sobel operator takes direction into account, it produces more stable and less noisy results compared to simpler methods (Gonzalez & Woods, 2018). This makes it especially useful in games for effects like object outlines, highlighting interactable elements, or stylized rendering such as toon shading. However, it is slightly more computationally expensive than simpler filters due to the multiple calculations involved.

Laplacian operator

The Laplacian operator is another edge detection method, but instead of looking at directional changes, it focuses on how much a pixel differs from its surrounding pixels overall (Gonzalez & Woods, 2018). It uses a single filter to detect areas where the rate of change itself changes rapidly.

Like Sobel, the Laplacian shader also samples neighboring pixels in a small grid, typically 3×3. However, instead of separating horizontal and vertical gradients, it compares the center pixel directly to its neighbors. If the difference is large, the pixel is likely part of an edge.

This approach allows the Laplacian operator to detect edges in all directions equally, often producing sharper and more defined results. However, because it is very sensitive to small changes, it can also pick up noise and fine details that may not be desirable in a game setting (Gonzalez & Woods, 2018). This can make the output look harsher or more cluttered if not properly filtered or combined with other techniques.

Despite this, the Laplacian operator is useful in cases where fine detail and strong edge definition are important, or when a more aggressive visual style is desired.

Method and Implementation

In this paper, both Sobel and Laplacian edge detection shaders will be implemented and tested within a custom rendering engine based on OpenGL. The engine is built from a provided handout from a previous module “Rendering Engines” and extended to support post-processing effects, user interface controls, and performance logging.

The goal of the testing is to compare how Sobel and Laplacian behave under different kernel sizes and rendering resolutions, and how these changes affect both visual output and performance.

Each shader will be tested using three kernel configurations: 3×3, 5×5, and 7×7. These variations are primarily used to evaluate how increasing the sampling range affects computational cost and overall rendering efficiency. While some minor observations will be made regarding visual differences, the main focus is on performance impact. In addition to this, the scene will be rendered at three different resolutions (400×300, 800×600, and 1600×1200) to measure how resolution scaling influences frame time and FPS under each shader and kernel configuration.

To ensure consistent and repeatable results, the test scene will consist of a single rotating cube. The rotation will be reset at the start of each capture to maintain identical starting conditions across tests. Each configuration will run for over 100 frames, during which performance data such as frame time (delta time) and frames per second (FPS) will be recorded and exported to a CSV file for later analysis.

User interface buttons will be used to switch between shaders. kernel sizes and to start the capture. This allows for efficient testing without restarting the application and ensures consistent comparison between configurations.

Additionally, a baseline measurement will be recorded for each resolution without any edge detection applied. This provides a reference point to evaluate the performance change introduced by both Sobel and Laplacian shaders.

Overall, this setup allows for a structured comparison of visual quality and performance trade-offs between the two edge detection methods under varying levels of detail and rendering load.

Results

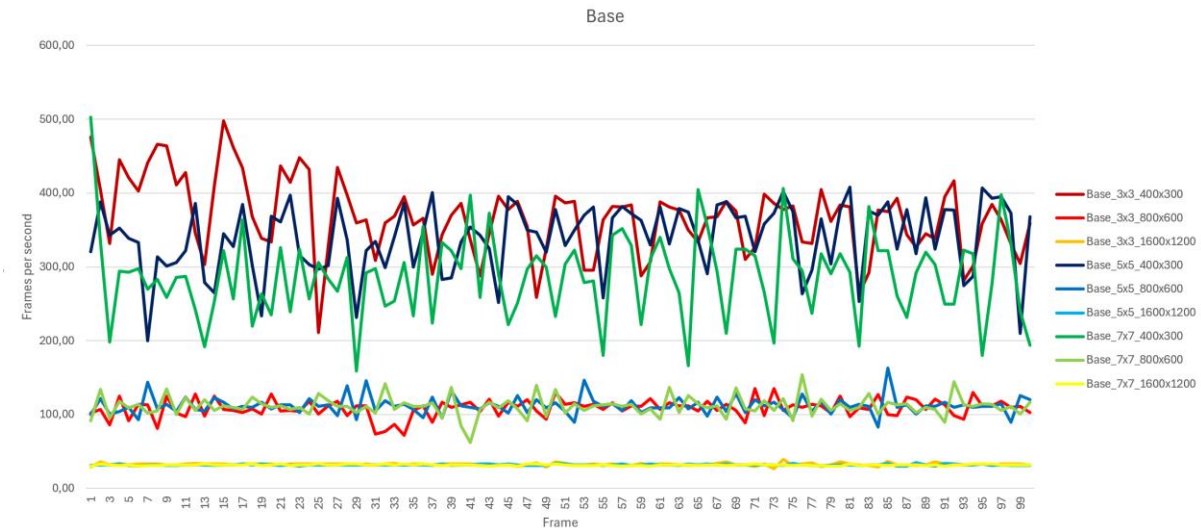


Figure 1: Graph of the base FPS

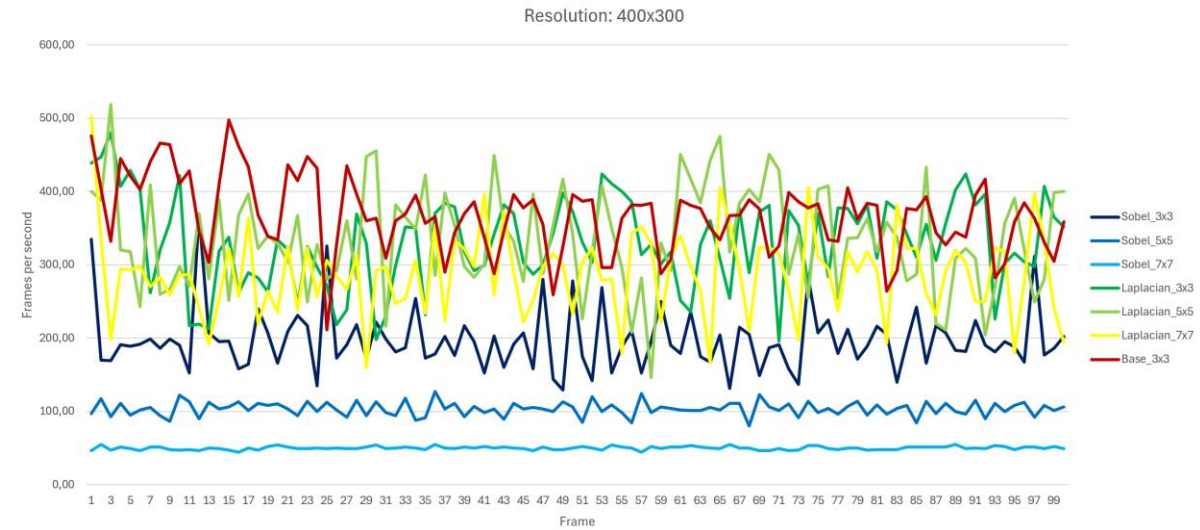


Figure 2: Graph of the resolution 400x300

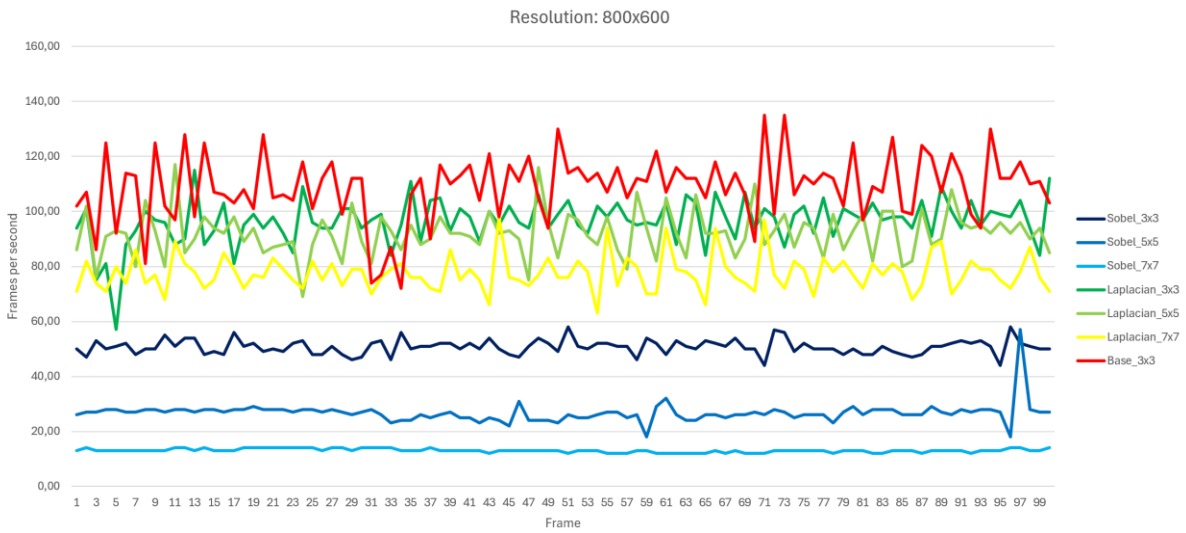


Figure 3: Graph of the resolution 800x600

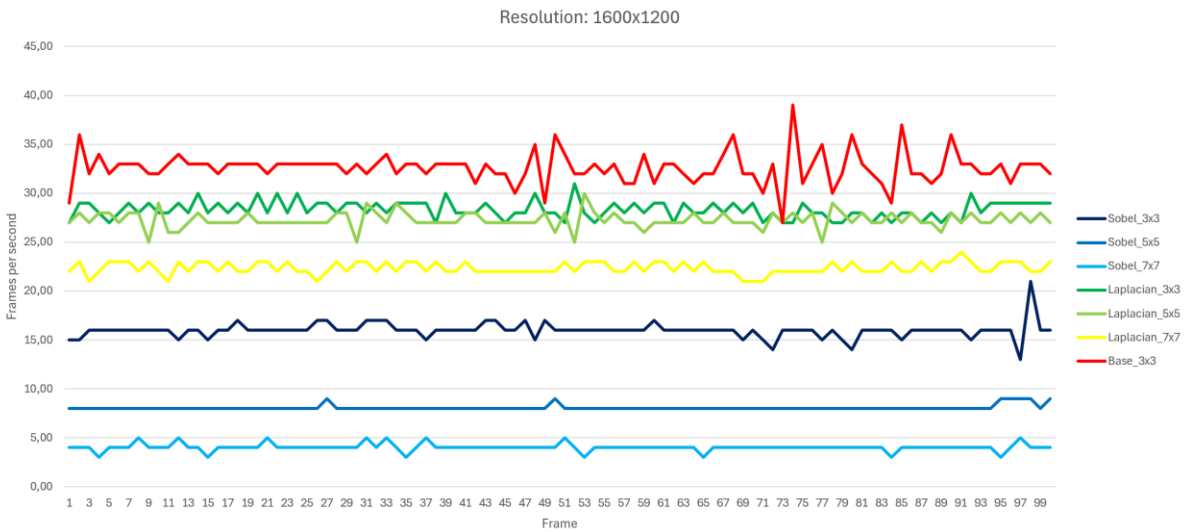


Figure 4: Graph of the resolution 1600x1200

Discussion

Looking at the graphs, the performance differences between the base renderer and the two edge detection shaders become clearly visible. The baseline (no edge detection) achieves the highest and most stable frame rate, averaging significantly higher FPS compared to all shader-based implementations.

Both Sobel and Laplacian shaders introduce a noticeable performance cost. The Laplacian implementation consistently performs better than Sobel across all kernel sizes, with Laplacian 3×3 achieving the highest FPS among the shader-based tests. As kernel size increases (5×5 and 7×7), a gradual decrease in performance can be observed for both methods due to the increased number of texture samples required per pixel.

The Sobel shader shows the heaviest performance impact overall. Even at 3×3, it runs slower than the heaviest Laplacian (7x7), and the gap becomes more clear at higher kernel sizes. This suggests that the Sobel operator is more computationally expensive in practice, as it requires calculating gradients in multiple directions and combining them to determine edge strength.

Despite fluctuations in frame-to-frame measurements, the overall trend remains consistent: baseline > Laplacian > Sobel in terms of performance, with kernel size having a direct negative correlation with FPS.

Complexity analysis

We can describe the performance of both Sobel and Laplacian edge detection shaders using Big-O notation by looking at how much work is done each frame.

Both shaders process the image in a fragment shader, meaning every pixel on the screen is handled individually. For an image with width W and height H, that's $W \times H$ pixels per frame.

For each pixel, the shader samples a neighborhood using a $k \times k$ kernel. This means it reads k^2texture values per pixel to calculate the final edge result.

Because of this, the overall time complexity per frame can be written as:

$O(W \times H \times k^2)$

This shows that performance mainly depends on two things:

1. Resolution $(W \times H)$
   Higher resolutions mean more pixels to process. If both width and height are doubled, the number of pixels becomes roughly four times larger, which directly increases frame time.
2. Kernel size (k^2)
   A larger kernel means more texture samples per pixel. For example, a 3×3 kernel uses 9 samples, while a 7×7 kernel uses 49, which significantly increases the work done per pixel.

Together, these factors multiply each other, which explains why both higher resolutions and larger kernels lead to lower FPS in the results. This means the complexity is multiplicative. The runtime scales linearly with the number of pixels and also linearly with the number of samples per pixel (kernel size squared). Because these factors multiply each other, increases in either resolution or kernel size have a combined effect on performance.

The measurements in the experiment match this fairly well. Increasing resolution consistently reduces FPS, and increasing kernel size also lowers performance across all resolutions. The fact that the performance curves look similar at each resolution also supports the idea that both shaders scale mainly based on pixel count and kernel size, as expected for a per-pixel post-processing effect.

Visual comparison

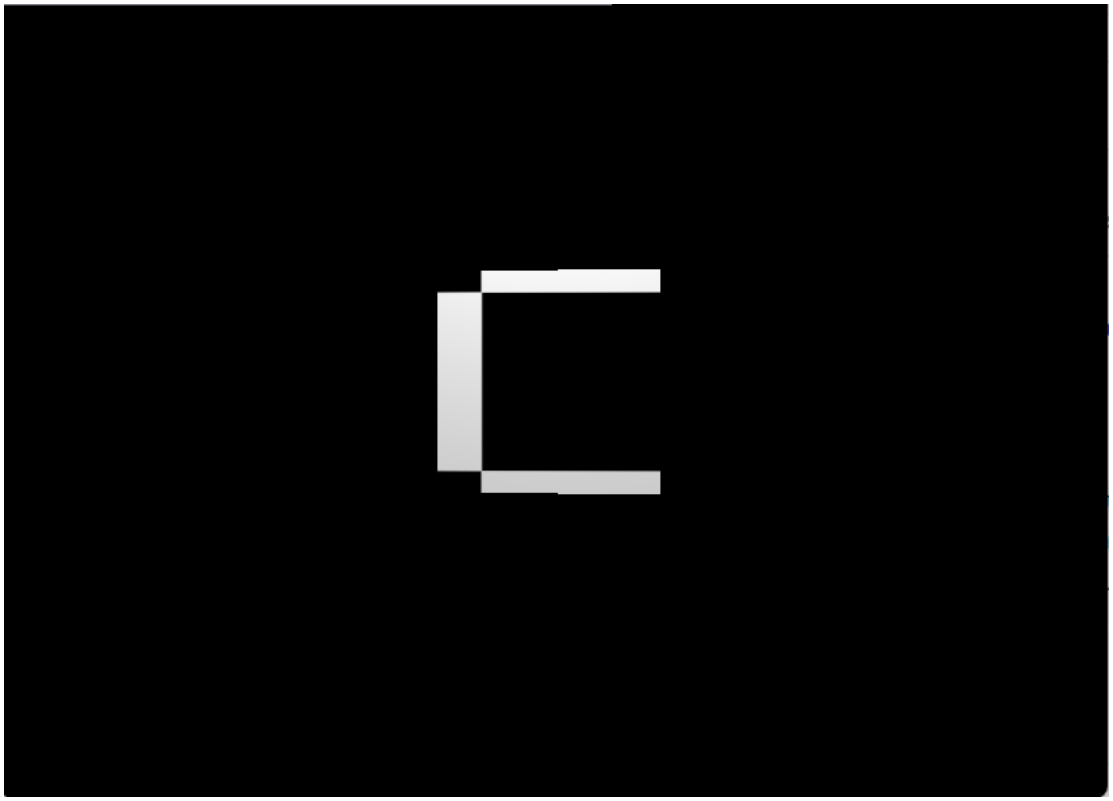


Figure 5: The base scene

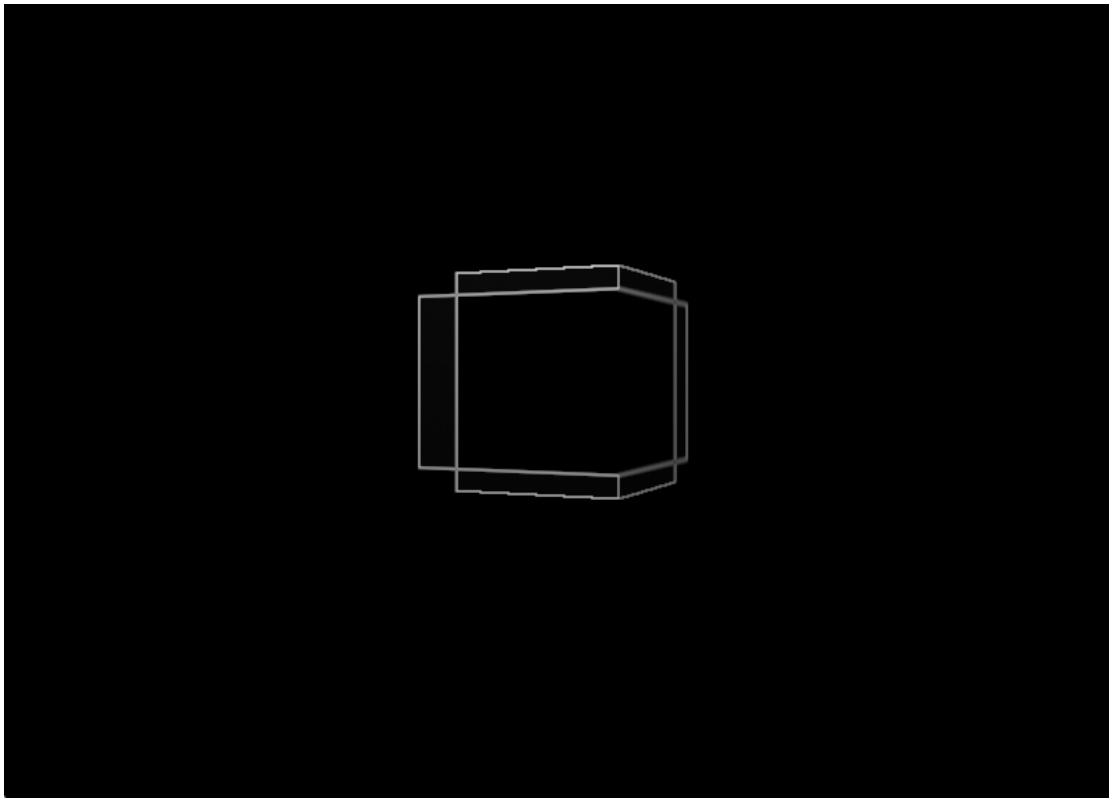


Figure 6: Sobel 3x3

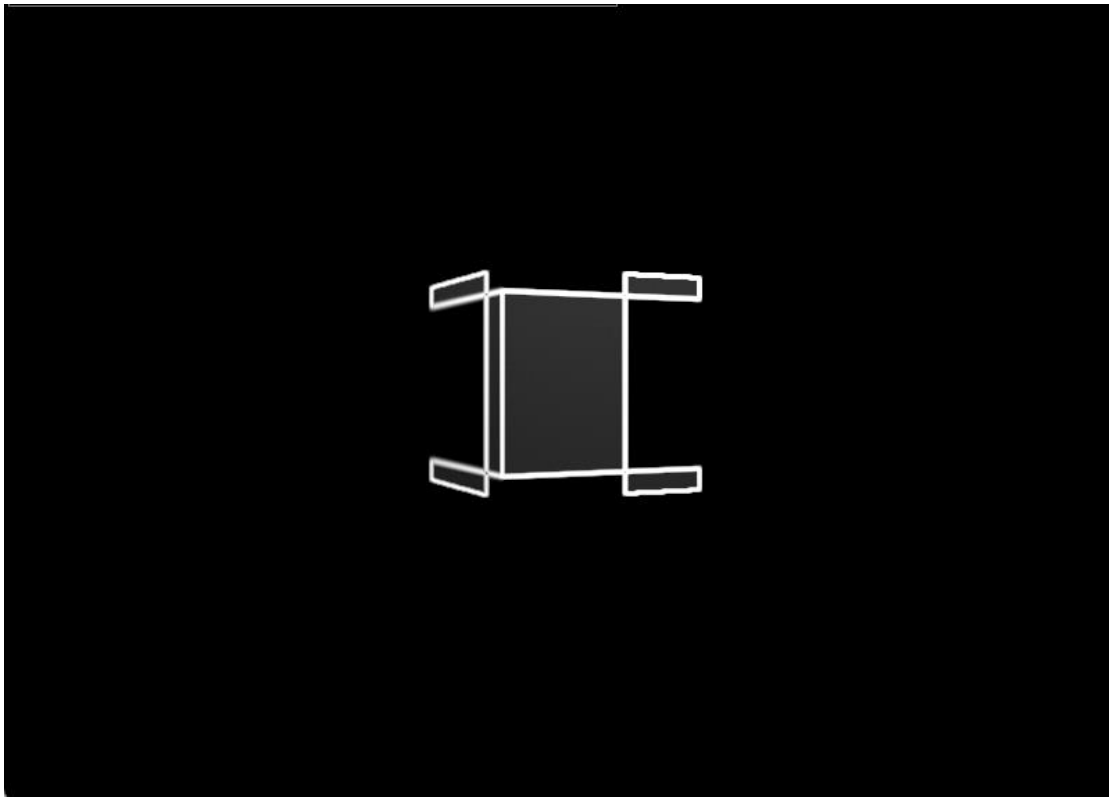


Figure 7: Sobel 5x5

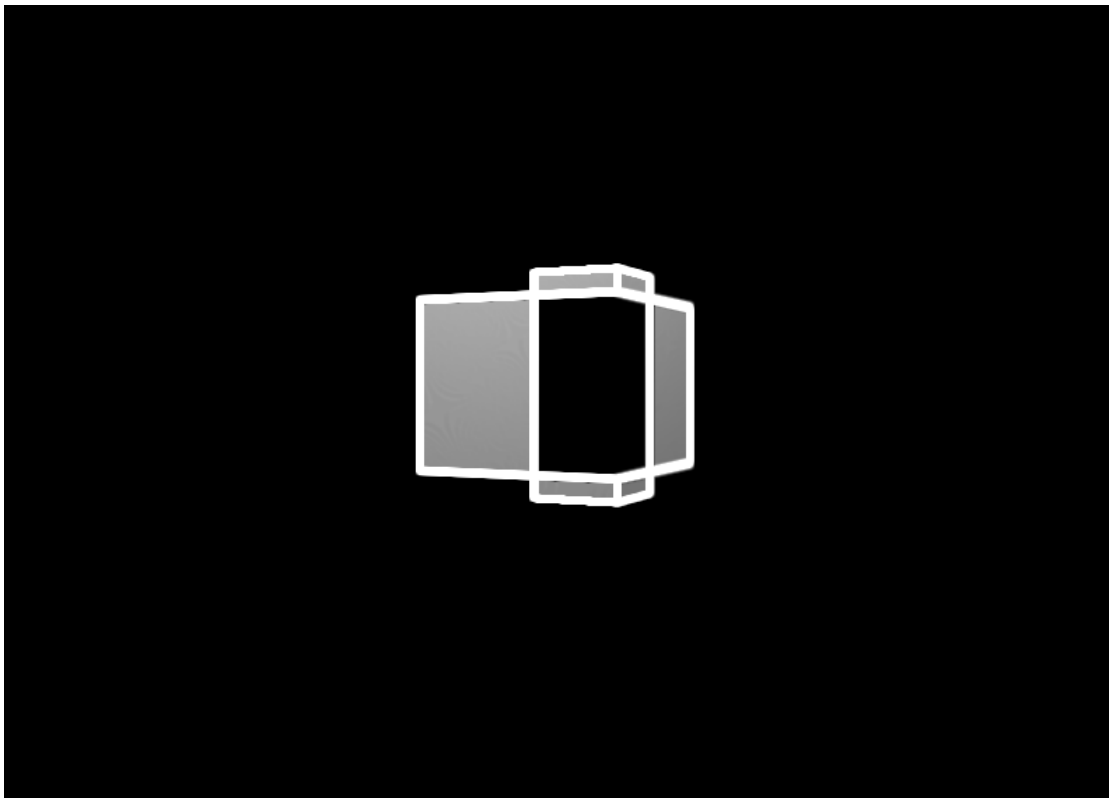


Figure 8: Sobel 7x7

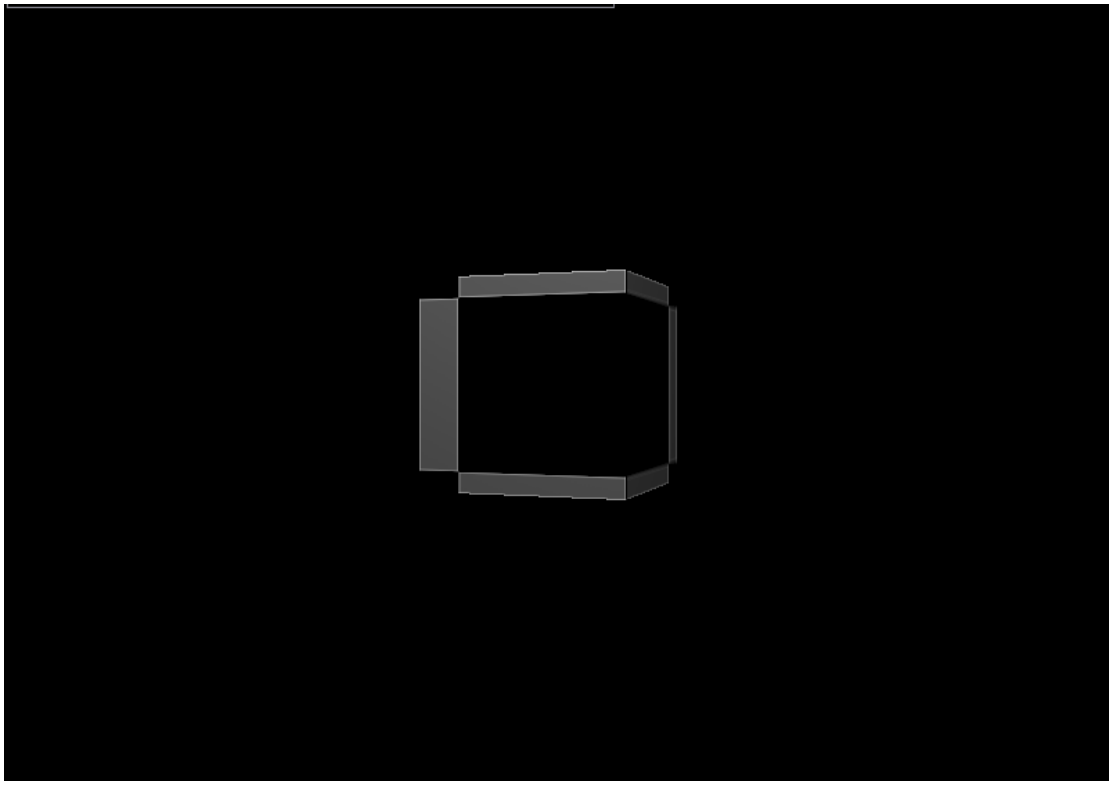


Figure 9: Laplacian 3x3

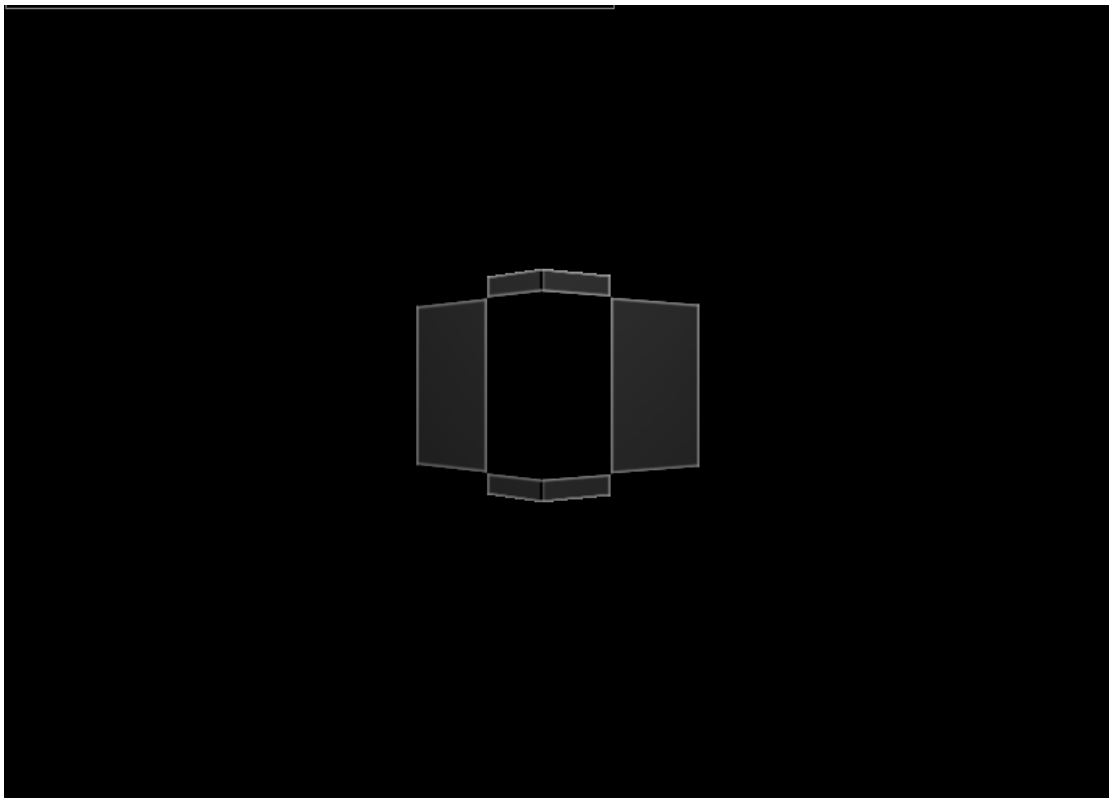


Figure 10: Laplacian 5x5

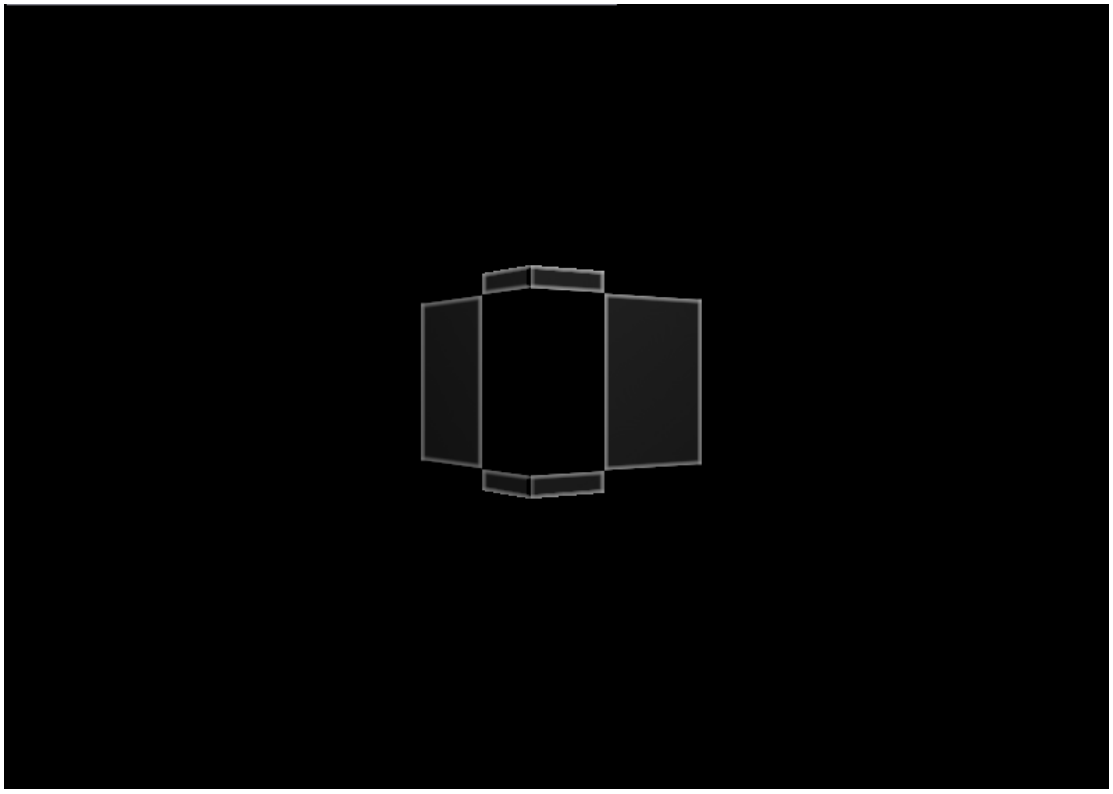


Figure 11: Laplacian 7x7

When comparing the visual output of both methods, Sobel clearly produces more readable and immediate edge results than Laplacian. Even at a 3×3 kernel, Sobel gives strong, continuous outlines that are easy to see and work well in real time. Laplacian at 3×3, on the other hand, is much weaker, a lot of edges are barely visible or feel incomplete, which makes it less useful on its own for clear in-game visuals. When increasing the Laplacian kernel size to 7×7, the edges become much more noticeable and start to look usable, but they still tend to feel a bit sharper and noisier compared to Sobel.

From a performance and complexity point of view, Laplacian is generally lighter. Even at 7×7, it still ends up being less expensive than Sobel at 3×3, which shows that Sobel has more cost per pixel due to combining directional gradients. This makes Laplacian more attractive when raw performance or higher FPS is the priority, especially if you can afford slightly weaker or noisier edge output.

Sobel, on the other hand, costs more but gives better results right away. You don't really need to increase its kernel size to get usable edges, since even 3×3 already looks clean and stable. That makes it more consistent for real-time applications where visual clarity matters more than squeezing out extra FPS.

Overall, Laplacian can be more efficient and scales better in terms of raw cost, but it needs larger kernels to become visually solid. Sobel is more expensive, but gives better-looking and more reliable edges at smaller configurations, which makes it the more practical choice when visual quality is the main goal.

Future improvements

One improvement would be to expand the shader implementations themselves. The current Sobel and Laplacian shaders could be optimized or extended. For example, Sobel could be implemented using a separable filter approach, reducing the number of texture samples. Laplacian could also be combined with a small blur (Gaussian pre-filter) to reduce noise and produce cleaner edges, especially at higher resolutions.

The experiment could also be improved by testing on more complex scenes. Using a single rotating cube makes results consistent, but it does not fully represent real game workloads. Adding scenes with multiple objects, different lighting conditions, and varying levels of detail would make the results more realistic and applicable to actual game development.

In addition, testing could be extended by including more kernel sizes and filter variations, or even comparing against other edge detection methods such as Prewitt or Canny-based approximations in shaders. This would provide a broader comparison of performance versus visual quality.

Finally, a useful improvement would be to include hardware and GPU profiling tools (such as OpenGL debug tools or GPU timers). This would give more precise insight into where the performance cost comes from, such as texture sampling, memory bandwidth, or shader instruction count.

Sources:

- Gonzalez, R. C., & Woods, R. E. (2018). *Digital image processing* (4th ed.). Pearson.
- Rost, R. J., Licea-Kane, B., Ginsburg, D., Sellers, G., & Kessenich, J. (2014). *The OpenGL shading language* (3rd ed.). Addison-Wesley.
- Sobel, I. (1970). *An isotropic 3×3 image gradient operator*. Stanford Artificial Intelligence Project (SAIL).
- Akenine-Möller, T., Haines, E., & Hoffman, N. (2018). *Real-time rendering* (4th ed.). A K Peters/CRC Press.